Projektuppgift

Programmering i C#.NET (DT071G)

Titel
Kvasirutt

Ziad Adeleo



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Institutionen för data och elektroteknik(DET)

Författare: Ziad Adeleo, ziba2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

Kvasirutt är en skrivbordsapplikation utvecklad i C# med hjälp av .NET-ramverket. Syftet med applikationen är att visualisera sökprocessen för två sökalgoritmer genom att visa antalet bearbetade noder och den beräknade väglängden mellan två rutor. Applikationen presenterar även sökresultaten i en resultattabell: algoritm, kartstorlek, väglängd, besökta noder och körtid.

Rapporten beskriver applikationens syfte och funktionalitet, den teoretiska bakgrunden till de tekniker och verktyg som används samt projektets planering såväl som genomförande. Slutligen beskrivs projektets resultat och slutsatser.

Innehållsförteckning

Sammanfattning **iii**

1 Introduktion **6**
1.1 Bakgrund och problemmotivering 6
1.2 Övergripande syfte 6
1.3 Detaljerad problemformulering 6
1.4 Mål 6
1.5 Avgränsningar 6
1.6 Översikt

2 Teori **7**
2.1 Figma 7
2.2 Visual Studio Code 7
2.3 Tillägg i Visual Studio Code 7
2.4 Programmeringsspråk och .NET 7
2.5 Windows Presentation Foundation (WPF) 7
2.6 Flödesschema 7
2.7 Sökalgoritmer 8
2.8 Git 8
2.9 GitHub 8

3 Metod **8**
3.1 Efterforskning 8
3.2 Krav 8
3.3 Design 8
3.4 Övergripande flöde 9
3.5 Programstruktur och arkitektur 9
3.6 Utveckling av applikation 9
3.7 Testing 9
3.8 Publicering till GitHub 9

4 Konstruktion **9**
4.1 Genomförd efterforskning och tillämpning 9
4.2 Framtagning av funktionella krav 10
4.3 Utformning av wireframes 11
4.4 Utformning av applikationens flöde 12
4.5 Strukturering av applikationen 13
4.6 Implementering och kodningsprinciper 13
4.7 Implementering av sökalgoritmer och visualisering 14
4.8 Kontinuerlig testning och förbättring 15
4.9 Versionshantering och publicering till GitHub 16
5 Resultat **16**
5.1 Applikationens slutresultat 16
5.2 Testresultat 16
5.3 Uppfyllelse av krav 17
6 Slutsatser **17**
6.1 Problem och utmaningar 17
6.2 Styrkor och svagheter 17
6.3 Förbättringar och vidareutveckling 17
6.4 Lärdomar 18
Källförteckning **19**
Bilaga A: Figma wireframe **20**
Bilaga B: Flödesschema **21**
Bilaga C: Kvasirutt (genomförd sökning) **22**

1 Introduktion

1.1 Bakgrund och problemmotivering

Kvasirutt är en desktopapplikation där sökalgoritmerna Breadth-First Search (BFS) och Dijkstras algoritm används för att hitta den kortaste vägen mellan en startpunkt och en målpunkt i en tvådimensionell karta. Applikationen visualiserar de besökta rutorna och den hittade sökvägen.

1.2 Övergripande syfte

Syftet är att utveckla en fungerande applikation med C# och Windows Presentation Foundation (WPF), där användaren kan välja olika start- och målpunkter samt se hur sökalgoritmerna söker efter en väg genom kartan. Applikationen ska även visa algoritmernas resultat.

1.3 Detaljerad problemformulering

Arbetet ska undersöka hur BFS och Dijkstras algoritm kan implementeras och visualiseras i en tvådimensionell karta. Det ska även visas hur algoritmernas olika sökstrategier påverkar vilka rutor som besöks och vilken väg som hittas.

1.4 Mål

Målet är att skapa en fungerande applikation som kan visualisera och visa BFS samt Dijkstras algoritm utifrån den hittade vägens längd, antalet besökta noder och algoritmens körtid.

1.5 Avgränsningar

Projektet avgränsas till en skrivbordsapplikation utvecklad med C# och WPF. Applikationen använder en tvådimensionell karta och fokuserar på BFS samt Dijkstras algoritm. Applikationen fokuserar på att visualisera och visa algoritmernas sökstrategier i ett rutnät.

För en mer detaljerad visualisering av grafens noder och kanter, för att tydligare visa algoritmernas sökstrategier utifrån grafteorins viktade- och oviktad grafer ingår inte i projektet.

1.6 Översikt

Rapporten beskriver först den teori och de verktyg som ligger till grund för projektet. Därefter presenteras metod och konstruktion, följt av

resultat samt en diskussion av applikationens funktion såväl som algoritmernas resultat.

2 Teori

2.1 Figma

Figma är ett designverktyg som används för att skapa prototyper eller utforma användargränssnitt utifrån applikationens krav [1].

2.2 Visual Studio Code

Visual Studio Code är en kodeditor som erbjuder funktioner för att utveckla program, organisera programfiler och navigera mellan dem [2].

2.3 Tillägg i Visual Studio Code

Ett tillägg i Visual Studio Code är ett program som utökar kodeditorns funktionalitet och kan ge stöd för olika programmeringsspråk. För att utveckla Kvasirutt i C# används tillägget C# Dev Kit tillsammans med .NET SDK. Detta ger verktyg för att bygga och köra .NET-applikationen [2].

2.4 Programmeringsspråk och .NET

C# är ett objektorienterat programmeringsspråk som används för att utveckla olika typer av applikationer. .NET är en utvecklingsplattform som erbjuder verktyg och bibliotek för att bygga samt köra applikationer med C#. Kombinationen av C# och .NET används för att utveckla Kvasirutt. Där C# hanterar programlogik och .NET tillhandahåller plattformen för applikationen [3].

2.5 Windows Presentation Foundation (WPF)

Windows Presentation Foundation (WPF) är ett ramverk inom .NET. Det används för att utveckla skrivbordsapplikationer med grafiska användargränssnitt för Windows. WPF använder XAML för att skapa gränssnittet och C# för att implementera applikationens funktionalitet [4].

2.6 Flödesschema

Ett flödesschema är en grafisk representation av en process där symboler och pilar används för att beskriva de olika stegen och deras ordningsföljd. Flödesscheman används inom programutveckling för att visuellt beskriva programmets logik, tydliggöra beslut och steg innan

implementationen påbörjas [5].

2.7 Sökalgoritmer

Sökalgoritmer används för att hitta vägar mellan noder i en graf. Olika algoritmer använder olika strategier för att söka efter en väg och avgöra vilka noder som ska undersökas. I Kvasirutt används Breadth-First Search (BFS) och Dijkstras algoritm för att visa sökningen samt vägen mellan en startpunkt såväl som en målpunkt [6, 7].

2.8 Git

Git är ett versionshanteringssystem som används för att registrera och hantera ändringar i projektfiler över tid. Systemet gör det möjligt att följa projektets utveckling, jämföra tidigare versioner och återställa filer vid behov. Git används i utvecklingen av Kvasirutt för att överföra en fungerande version till GitHub [8].

2.9 GitHub

GitHub är en plattform som används för att lagra och hantera projekt med hjälp av Git. Plattformen gör det möjligt att samla projektets filer och ändringshistorik i ett repository. GitHub används i utvecklingen av Kvasirutt för att lagra källkoden och hålla projektets filer tillgängliga [9].

3 Metod

3.1 Efterforskning

Arbetet inleddes med efterforskning om sökalgoritmer, C# och utveckling av grafiska applikationer. Informationen användes för att välja algoritmer och tekniker för Kvasirutt.

3.2 Krav

Utifrån efterforskningen sammanställdes funktionella krav (se 4.2) för att definiera applikationens funktioner, användarinteraktioner och felhantering. Kraven användes som underlag för design och implementation.

3.3 Design

En wireframe skapades i Figma för att planera gränssnittets layout och

placering av funktioner innan utvecklingen påbörjades. Se bilaga A.

3.4 Övergripande flöde

Ett flödesschema togs fram utifrån krav och design för att beskriva applikationens huvudsakliga arbetsflöde samt fungera som stöd under implementationen. Se bilaga B.

3.5 Programstruktur och arkitektur

Programstrukturen planerades utifrån Separation of Concerns, med målet att skilja sökalgoritmerna från kartmodellen och användargränssnittet. Single responsibility principle tillämpades för att göra koden tydligare och enklare att underhålla [10].

3.6 Utveckling av applikationen

Utvecklingen genomfördes stegvis utifrån kraven, wireframen och flödesschemat. Först skapades gränssnittet därefter kartmodellen, sökalgoritmerna och slutligen kopplades funktionerna samman med gränssnittet och visualiseringen.

3.7 Testning

Applikationen testades löpande för att kontrollera funktionaliteten, upptäcka fel och säkerställa att ändringar inte påverkade redan fungerande delar.

3.8 Publicering till GitHub

Git användes för att publicera en fungerande version av Kvasirutt till GitHub, för att publicera källkoden. En .gitignore-fil användes för att inte ta med genererade filer och mappar som inte behövde publiceras.

4 Konstruktion

4.1 Genomförd efterforskning och tillämpning

Efterforskningen fokuserade på sökalgoritmerna Breadth-First Search (BFS) och Dijkstra samt hur de kan användas för att hitta vägar genom ett rutnät. BFS undersöktes som en algoritm som söker efter den

kortaste vägen räknat i antal steg när alla förflyttningar har samma kostnad. Dijkstras algoritm undersöktes för att förstå hur vikter kan användas för att beräkna den billigaste vägen när olika celler har olika kostnader (heltal mellan 1 och med 5 syns inte på kartan).

Kunskapen från efterforskningen användes för att välja och implementera de två algoritmerna i Kvasirutt. För att kunna jämföra deras sökstrategier tilldelades kartans celler vikter. BFS använder inte dessa vikter vid sökningen, medan Dijkstra tar hänsyn till dem när den beräknar vägen. Detta gör det möjligt att undersöka hur algoritmernas olika strategier påverkar den väg som hittas.

Efterforskningen omfattade även tekniker för att utveckla ett grafiskt användargränssnitt i C# och .NET. WPF valdes för att skapa en skrivbordsapplikation där sökningen kan visualiseras direkt i ett rutnät. Denna lösning gör det möjligt för användaren att följa algoritmernas sökprocess och jämföra resultat, bland annat väglängd, antal besökta noder och körtid.

4.2 Framtagning av funktionella krav

Utifrån efterforskningen och projektets syfte sammanställdes en kravspecifikation för Kvasirutt. Kraven omfattar applikationens funktioner, hantering av användarens interaktioner, visualisering av sökprocessen och presentation av resultat. De omfattar även felhantering för att förhindra att sökningen startas när nödvändiga förutsättningar inte är uppfyllda. Kravspecifikationen användes som underlag för utformningen av användargränssnittet och implementationen av applikationen. Se tabell nedan.

Krav	**Beskrivning**
Välja kartstorlek	Användaren ska kunna välja kartans storlekarna 20x20, 30x30 och 40x40
Visa rutnät	Visa kartan som ett tvådimensionellt rutnät
Ange startpunkt	Användaren ska kunna placera en startpunkt på kartan.
Ange målpunkt	Användaren ska kunna placera en målpunkt på kartan.
Flytta startpunkt	Användaren ska kunna flytta

	startpunkt.
Flytta målpunkt	Användaren ska kunna flytta målpunkt.
Välja sökalgoritm	Användaren ska kunna välja sökalgoritm.
Starta sökning	Användaren ska kunna starta sökningen med den valda algoritmen.
Stoppa sökning	Användaren ska kunna stoppa en pågående visualisering
Visualisera sökprocessen	Visa de besökta noderna/rutorna
Visualisera hittad väg	Markera vägen från startpunkten till målpunkten när en väg hittas.
Visa sökresultat	algoritm, kartstorlek, väglängd, antal besökta noder och körtid
Uppdatera sökresultat	uppdateras när användaren genomför en ny sökning
Återställ visualisering	användarens drar i en ruta eller skapar en ny karta under sökprocessen
Felhantering	visualisering får endast starta om: kartstorlek, algoritm, startpunkt och målpunkt har valts. Om inte visa felmeddelande

4.3 Utformning av wireframe

Wireframen designades i Figma utifrån kravspecifikationen för att planera användargränssnittets layout och placering av funktioner innan implementationen påbörjades. Gränssnittet delades in i tre huvudsakliga delar: kontrollpanelen, karta och resultat.

Kontrollpanelen placerades till höger i gränssnittet och innehåller inställningar för kartstorlek, val av sökalgoritm samt start- och målpunkt. Där placerades även knapparna Skapa karta och Starta visualisering, så att användaren kan skapa en karta utifrån valda inställningar och därefter starta sökprocessen.

Till vänster placerades kartvyn, som visar rutnätet och de individuella cellerna. Startpunkten och målpunkten markeras i rutnätet för att

tydliggöra var sökningen börjar och vilket mål algoritmen ska försöka nå. Kartan utgör även den visuella ytan där algoritmens sökprocess och den hittade vägen presenteras.

Under kartvyn placerades resultatsektionen med en tabell för att presentera information om den genomförda sökningen. Tabellen innehåller kolumnerna **Algoritm**, **Kartstorlek**, **Väglängd**, **Besökta noder** och **Körtid**. Detta gör det möjligt att se både vilken algoritm som användes och hur sökningen gick, utifrån de beräknade värdena.

Genom att samla inställningarna i kontrollpanelen, visualiseringen i kartvyn och resultatet i en separat tabell skapades en tydlig struktur där användaren kan göra val för sökningen, följa dess förlopp och ta del av resultatet.

Wireframen (se bilaga A) användes sedan som underlag vid implementeringen av det grafiska gränssnittet i WPF.

4.4 Utformning av applikationens flöde

För att visa hur Kvasirutt ska fungera skapades ett flödesschema utifrån kravspecifikationen och wireframen. Flödesschemat beskriver applikationens huvudsakliga arbetsflöde, från att användaren väljer inställningar och skapar en karta till att sökningen genomförs och resultatet visas.

Flödet börjar med att användaren väljer kartstorlek och sökalgoritm samt anger startpunkt och målpunkt. När användaren väljer att skapa kartan byggs rutnätet utifrån den valda storleken. Därefter kan användaren starta visualiseringen av sökningen.

Innan sökalgoritmen körs kontrollerar applikationen att nödvändiga krav är uppfyllda, exempelvis att startpunkt och målpunkt har angetts. Om kraven inte är uppfyllda ska sökningen inte startas. Om kontrollerna godkänns körs den valda algoritmen, var sökprocessen visualiseras genom att besökta celler och den hittade vägen markeras i rutnätet.

När sökningen är färdig presenteras resultatet i resultattabellen. Där visas den valda algoritmen, kartstorleken, väglängden, antalet besökta noder och körtiden.

Flödesschemat användes som stöd under implementationen för att strukturera programmets logik och tydliggöra ordningen mellan de olika stegen. Flödesschemat visas i bilaga B.

4.5 Strukturering av applikationen

För att göra koden mer strukturerad delades applikationen upp i mappar utifrån olika ansvarsområden. Syftet var att skilja sökalgoritmerna från representationen av kartan och sökresultatet samt från det grafiska användargränssnittet.

Mappen Algorithms innehåller klasserna Bfs och Dijkstar samt interfacet IPathfindingAlgorithm. Interfacet definierar metoden FindPath, som båda algoritmerna implementerar. Detta innebär att algoritmerna följer samma grundläggande kontrakt för att genomföra en sökning, trots att de använder olika strategier.

Mappen Pathfinding innehåller de klasser och typer som används för att representera kartan och sökresultatet. Klassen Map hanterar kartans storlek, celler och start- och målpositioner. Klassen Cell representerar en enskild cell och innehåller information om dess vikt och typ. Position används för att identifiera en cell genom dess rad- och kolumn koordinater, medan CellType anger om cellen är tom, startpunkt eller målpunkt. PathfindingResult samlar information om sökresultatet, bland annat den hittade vägen, besökta positioner, väglängd och antal besökta noder.

Det grafiska gränssnittet definieras i MainWindow.xaml, medan MainWindow.xaml.cs hanterar användarinteraktioner och kopplar samman gränssnittet med applikationens funktionalitet. Där hanteras bland annat skapandet av rutnätet, val av algoritm, placering och förflyttning av start- och målpunkt, visualisering av sökningen och presentation av resultatet.

Strukturen innebär att kartans representation och sökalgoritmerna ligger i separata delar från gränssnittets kod. Det gör ansvarsområdena tydligare och underlättar förståelsen och vidareutvecklingen av applikationen. Samtidigt samordnar MainWindow.xaml.cs flera av programmets funktioner och fungerar som kopplingen mellan användarens interaktioner och söklogiken.

4.6 Implementering och kodningsprinciper

Applikationen utvecklades i C# med .NET och Windows Presentation Foundation (WPF). Gränssnittets layout definierades i XAML, medan C# användes för att implementera funktionaliteten och hantera användarens interaktioner.

Vid implementationen eftersträvades en ansvarsfördelning enligt principen Separation of Concerns. Detta innebär att applikationens olika

delar organiserades efter sina huvudsakliga ansvarsområden. Sökalgoritmerna placerades i mappen Algorithms, medan kartans representation och sökresultatets klasser placerades i Pathfinding. Gränssnittet och hanteringen av användarens interaktioner implementerades i MainWindow.xaml och MainWindow.xaml.cs.

Även principen Single Responsibility Principle (SRP) användes som riktlinje vid utformningen av klasserna. SRP innebär att en klass bör ha ett huvudsakligt ansvar och därmed en huvudsaklig anledning att förändras. Exempelvis ansvarar Position för att representera en position genom rad- och kolumnkoordinater, medan PathfindingResult samlar information om resultatet från en sökning. På motsvarande sätt ansvarar Bfs och Dijkstar för att implementera varsin sökalgoritm.

För att skapa ett gemensamt sätt att anropa algoritmerna definierades interfacet IPathfindingAlgorithm med metoden FindPath. Båda algoritmerna implementerar samma kontrakt, vilket gör att gränssnittet kan använda dem på ett gemensamt sätt. Algoritmerna arbetar med kartans datamodell i stället för att direkt hantera grafiska komponenter. Resultatet returneras genom PathfindingResult och används därefter av gränssnittet för visualisering och presentation av resultat.

Felhantering vid implementationen. Applikationen behöver kontrollera att nödvändiga förutsättningar, exempelvis att startpunkt och målpunkt finns, är uppfyllda innan sökningen startas. Detta minskar risken för att felaktiga tillstånd leder till fel under körning.

Principerna användes som riktlinjer för att göra koden mer begriplig, underhållbar och möjlig att vidareutveckla. Samtidigt behölls lösningen på en nivå som var rimlig för projektets omfattning.

4.7 Implementering av sökalgoritmer och visualisering

I Kvasirutt implementerades två sökalgoritmer: Breadth-First Search (BFS) och Dijkstras algoritm. Båda används för att hitta en väg från startpunkten till målpunkten i kartans rutnät, men de skiljer sig åt i hur de väljer nästa position att undersöka.

BFS söker igenom kartan nivå för nivå och använder en kö för att hantera vilka positioner som ska undersökas härnäst. Algoritmen hittar en kortaste väg räknat i antal förflyttningar när alla förflyttningar betraktas som lika kostsamma. Den tar därför inte hänsyn till cellernas vikter.

Dijkstras algoritm tar hänsyn till vikterna i kartans celler. Varje cell tilldelas en vikt som används för att beräkna den sammanlagda

kostnaden för att nå en position. Algoritmen håller reda på den hittills lägsta kända kostnaden för varje position och uppdaterar den när en “billigare” väg hittas. På så sätt kan algoritmen hitta en väg med lägst sammanlagd kostnad, även om den inte har minst antal förflyttningar räknat i antalet steg.

För att algoritmerna skulle kunna arbeta med samma karta implementerades de med det gemensamma interfacet IPathfindingAlgorithm. Båda använder metoden FindPath, som tar emot kartan samt start- och målpunkten. Under sökningen samlas information om de besökta positionerna och vilka positioner som föregår andra längs den hittills bästa vägen. Denna information används för att återskapa vägen när sökningen har nått målet.

Sökresultatet returneras i ett objekt av typen PathfindingResult. Objektet innehåller den hittade vägen, de besökta positionerna, väglängden och antalet besökta noder. Dessa värden används både för visualiseringen och för presentation av resultat i gränssnittet.

När användaren startar visualiseringen anropas den valda algoritmen. Körtiden mäts med Stopwatch och används som ett av värdet i resultattabellen. Därefter visas sökprocessen stegvis i rutnätet genom att de besökta cellerna markeras med grå färg. När dessa har visualiserats markeras den hittade vägen från startpunkten till målpunkten med cyan färg. Fördröjningar mellan markeringarna gör att användaren kan följa sökprocessen visuellt i stället för att endast se slutresultatet i tabellen.

Slutligen presenteras sökresultatet i resultattabellen med algoritmens namn, kartstorlek, väglängd, antal besökta noder och körtid. På detta sätt kombineras algoritmernas beräkningar med en grafisk visualisering som gör deras sökprocess och resultat lättare att följa.

4.8 Kontinuerlig testning och förbättring

Testningen genomfördes kontinuerligt under utvecklingen av Kvasirutt för att kontrollera att applikationens funktioner fungerade som tänkt och att nya ändringar inte orsakade problem i redan implementerad funktionalitet.

Testningen omfattade bland annat skapandet av kartor i olika storlekar, placering och förflyttning av start- och målpunkt samt körning av BFS och Dijkstras algoritm. Sökresultaten kontrollerades för att undersöka om algoritmerna hittade vägar och om resultatmåtten, såsom väglängd och antal besökta noder, presenterades i gränssnittet.

Även användargränssnittets beteende kontrollerades under utvecklingen. Detta omfattade att funktionerna kunde användas i rätt

ordning och att visualiseringen och resultattabellen uppdaterades vid nya sökningar. Fel och oönskat beteende som upptäcktes under testningen åtgärdades.

Genom att testa applikationen under utvecklingens gång kunde problem upptäckas innan fler funktioner byggdes ovanpå den befintliga koden. Testningen bidrog därmed till att förbättra applikationens funktionalitet och stabilitet.

4.9 Versionshantering och publicering till GitHub

När en fungerande version av applikationen hade testats sparades ändringarna i Git och skickades till GitHub.

En .gitignore-fil skapades för att utesluta genererade filer och mappar som inte behövde ingå i projektets publicerade källkod. Detta bidrog till att hålla GitHub repot fokuserat på de filer som behövs för att förstå och bygga projektet.

Projektet publicerades till GitHub för att göra källkoden tillgänglig för lärarna vid bedömningen.

5 Resultat

5.1 Applikationens slutresultat

Projektet resulterade i Kvasirutt, en skrivbordsapplikation utvecklad i C# med .NET och WPF. Användaren kan skapa tre olika rutnät, välja sökalgoritm samt placera och flytta start- och målpunkt. Sökprocessen visualiseras genom att besökta celler och den hittade vägen markeras i rutnätet.

Två sökalgoritmer implementerades: Breadth-First Search (BFS) och Dijkstras algoritm. BFS söker efter en kortaste väg räknat i antal steg, medan Dijkstra tar hänsyn till cellernas vikter för att hitta en väg med lägst sammanlagd kostnad.

Efter sökningen presenteras algoritm, kartstorlek, väglängd, antal besökta noder och körtid i en resultattabell. Applikationens gränssnitt och genomförd sökning visas i Bilaga C.

5.2 Testresultat

Under utvecklingen testades skapandet av kartor i olika storlekar, val av sökalgoritm samt placering och förflyttning av start- och målpunkt. Även körningen av BFS och Dijkstra, visualiseringen av besökta celler och

den hittade vägen samt uppdateringen av resultattabellen vid nya sökningar kontrollerades.

Testningen användes för att kontrollera att applikationens funktioner fungerade tillsammans och att sökresultaten presenterades i gränssnittet.

5.3 Uppfyllelse av krav

Kvasirutt uppfyller projektets krav genom att erbjuda en fungerande skrivbordsapplikation för att visualisera processen av BFS och Dijkstra. De centrala funktionerna består av kartskapande, val av algoritm, placering och förflyttning av start- och målpunkt samt visualisering och presentation av sökresultat.

Programstrukturen bygger på ett gemensamt interface för algoritmerna och en uppdelning mellan söklogik, kartmodell och gränssnitt.

6 Slutsatser

6.1 Problem och utmaningar

En utmaning var att mycket funktionalitet samlades i MainWindow.xaml.cs. En tidigare refaktorering av kod hade kunnat ge en tydligare struktur och enklare underhåll.

En annan utmaning var att presentera sökalgoritmernas resultat med den stegvisa visualiseringen och säkerställa att resultatet presenterades korrekt.

6.2 Styrkor och svagheter

En styrka med Kvasirutt är att applikationen visualiserar BFS och Dijkstra och presenterar resultat som gör det möjligt att undersöka deras sökningar. Uppdelningen mellan algoritmerna och kartmodellen bidrar till en tydligare programstruktur.

En svaghet är att MainWindow.xaml.cs fortfarande har ett stort ansvar. Genom att flytta delar av logiken till separata klasser hade koden kunnat bli enklare att underhålla och vidareutveckla.

6.3 Förbättringar och vidareutveckling

Kvasirutt skulle kunna vidareutvecklas med grafer för att jämföra algoritmernas resultat och med fler sökalgoritmer, exempelvis A*. Olika

grafer skulle kunna användas för att tydligare demonstrera skillnaden mellan oviktad sökning och sökning med viktad graf.

Felmeddelanden skulle dessutom kunna inplementeras direkt i gränssnittet i stället för att visas med MessageBox, vilket troligtvis skulle ge en bättre användarupplevelse.

6.4 Lärdomar

Projektet visade betydelsen av att planera programstrukturen och fördela ansvaret mellan olika delar innan implementationen växer. Principer som Separation of Concerns och Single Responsibility Principle kan bidra till en mer begriplig och underhållbar kod, men behöver följas upp genom löpande refaktorering.

En annan lärdom är vikten av att testa funktioner kontinuerligt och balansera ny funktionalitet mot kodkvalitet. En applikation behöver inte bara fungera, utan bör också vara strukturerad så att den går att förstå, testa och vidareutveckla.

Källförteckning

[1] Figma, ”What is Figma?” https://help.figma.com/hc/en-us/articles/14563969806359-What-is-Figma Publicerad datum saknas. Hämtad 2026-10-13

[2] Visual Studio Code, ”The open source code editor”, https://code.visualstudio.com/ Publicerad datum saknas. Hämtad 2026-10-13.

[3] Microsoft, ”Overview - A tour of C# | Microsoft Learn”. https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview Publicerad datum saknas. Hämtad 2026-10-13.

[4] Microsoft, “What is Windows Presentation Foundation - WPF | Microsoft Learn”. *https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/* Publicerad datum saknas. Hämtad 2026-10-13.

[5] IBM, ”What is a Flowchart?”, https://www.ibm.com/think/topics/flowchart Publicerad datum saknas. Hämtad 2026-10-13.

[6] Algorithms for Competitive Programming, “Breadth First Search” *https://cp-algorithms.com/graph/breadth-first-search.html* Publicerad 2026-09-18. Hämtad 2026-10-13.

[7] Wikipedia, “Dijkstar’s algorithm” https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm Publicerad datum saknas. Hämtad 2026-10-13.

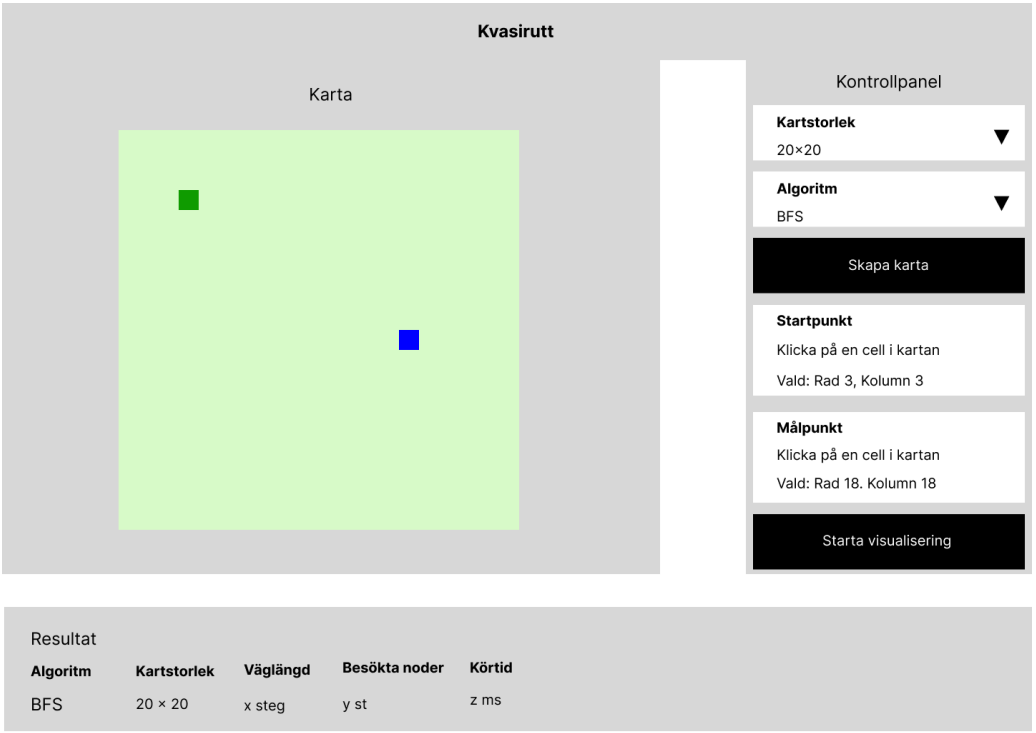
[8] Git, “What is Git?”, https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F Publicerad datum saknas. Hämtad 2026-10-13.

[9] GitHub, “About repositories - GitHub Docs” https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories Publicerad datum saknas. Hämtad 2026-10-13.

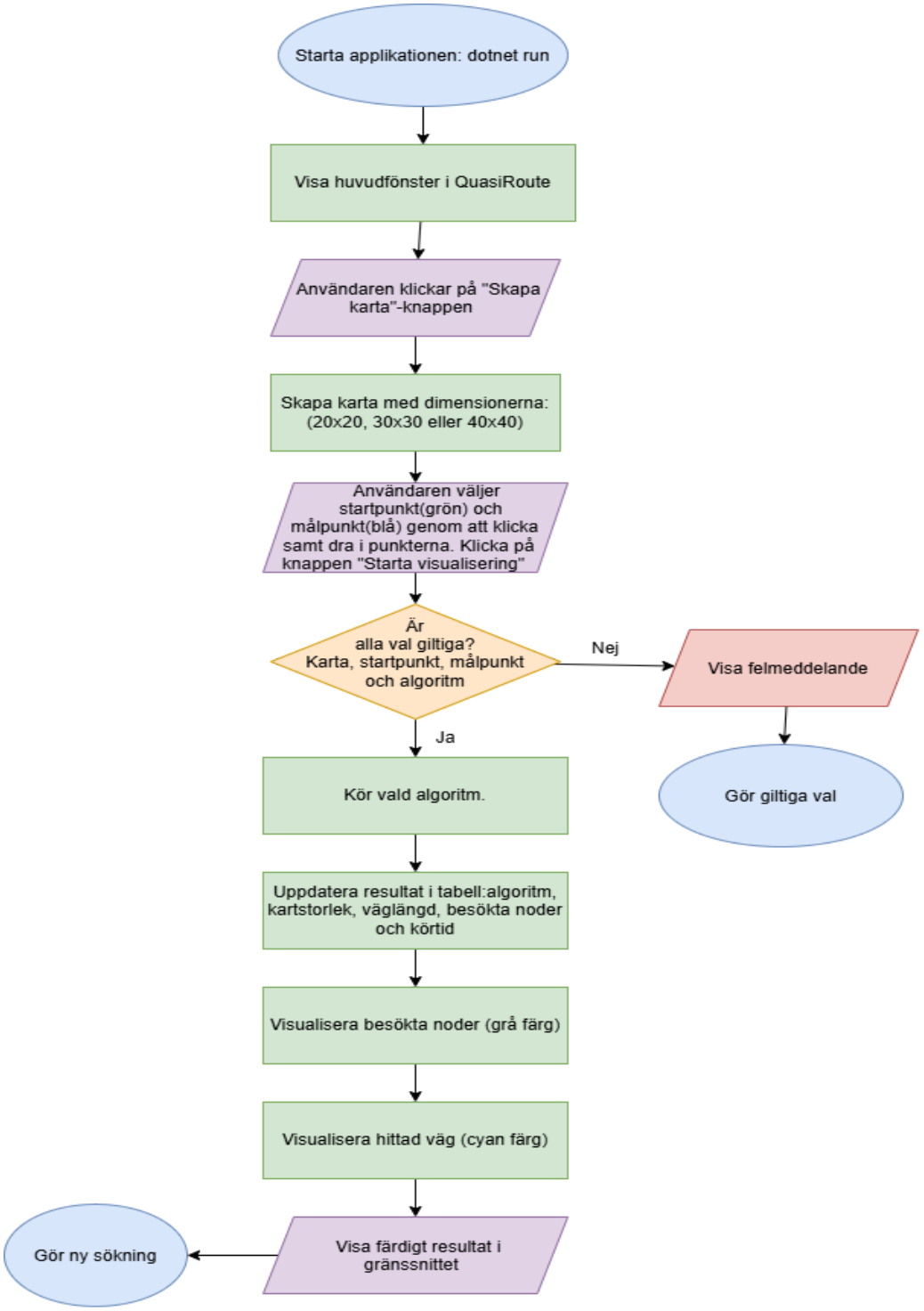
[10] Hassan Mohsen, "Single responsibility and Separation of concerns principles” https://developer.onepagecrm.com/blog/single-responsibility-and-separation-of-concerns-principles/ Publicerad 2022-07-18. Hämtad 2026-10-15.

[11]

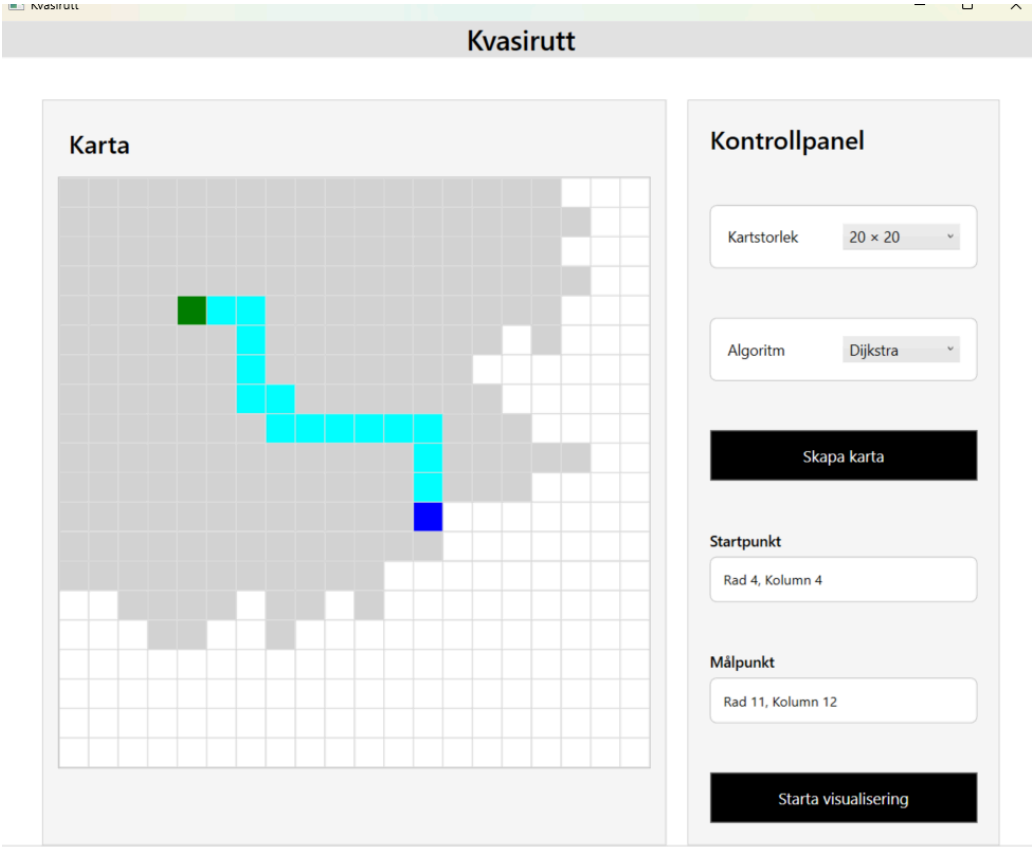
Bilaga A: Figma wireframe



Bilaga B: Flödesschema (QuasiRoute ändrades till Kvasirutt)



Bilaga C: Kvasirutt (genomförd sökning)



Resultat

Algoritm	Kartstorlek	Väglängd	Besökta noder	Körtid
Dijkstra	20 × 20	15	229	70,23 ms